Trace Performance Analysis

Key Takeaways

This document benchmarks MLflow tracing across ingestion, search, client-side serialization, runtime overhead, and backend choice. The goal is to identify the dominant bottlenecks, quantify their impact, and prioritize fixes.

Top findings

1. **Ingestion is bottlenecked by ORM write amplification.** `log_spans()` uses `session.merge()` per span and per span metric. This dominates CPU time, scales linearly with span count, and caps SQLite throughput around 550-600 spans/s.
2. **Search is bottlenecked by N+1 relationship loading.** `search_traces()` executes one base query, then lazy-loads tags, metadata, and assessments per result row. For 100 results this becomes 301 queries, which dominates simple search latency and hurts PostgreSQL even more.
3. **A few specialized paths have isolated bottlenecks.** Full-text search scans JSON blobs, assessment-filtered search is much slower than baseline search, disabled tracing is not zero-cost, and streaming tracing adds measurable per-chunk overhead.

Recommended actions

1. **Bulk insert spans and span metrics in `log_spans()`.**
2. **Eager-load trace relationships in `search_traces()`.**
3. **Add an early disabled fast path in `@mlflow.trace`.**
4. **Replace unindexed span content scanning with indexed lookup** for full-text search.
5. **Address assessment-search indexing and span-processor lock contention.**

What is not the bottleneck

- HTTP transport overhead is negligible relative to store-layer work.
- The async export queue is not a bottleneck.
- Large span payloads mainly affect ingestion and storage size, not baseline search latency.

Scope And Methodology

Workloads covered

- Ingestion latency and throughput
- Search latency across common query patterns
- Sustained write load
- Client-side serialization cost
- Concurrent writers
- Assessment CRUD and filtered search
- Large-content traces
- Runtime overhead of tracing decorators and streaming
- Span-processor lock contention
- Async export batching
- End-to-end OTLP HTTP path
- SQLite vs PostgreSQL comparison

Test setup

- Store: `SqlAlchemyStore` directly unless otherwise noted

- Primary backend: SQLite in a temporary local directory
- Secondary backend: PostgreSQL 16 in local Docker
- Data: synthetic traces with realistic span distributions
- Search default: `max_results=100`
- Sustained load duration:
  - SQLite: 60s per config
  - PostgreSQL: 30s per config

Caveats

- The sustained-load explanation is partly inferred from profiling of the ingestion path rather than independently profiled end-to-end.
- The attribution of simple search latency to lazy loading is strongly supported by query-count behavior, but is still an inference rather than a profiler-backed call tree.
- Client-side serialization costs matter primarily for large payloads; for typical small traces, server-side bottlenecks dominate.

Ingestion

Takeaway

Ingestion scales roughly linearly with span count because the hot path performs ORM work per span. Payload size matters much less than span count for the storage layer.

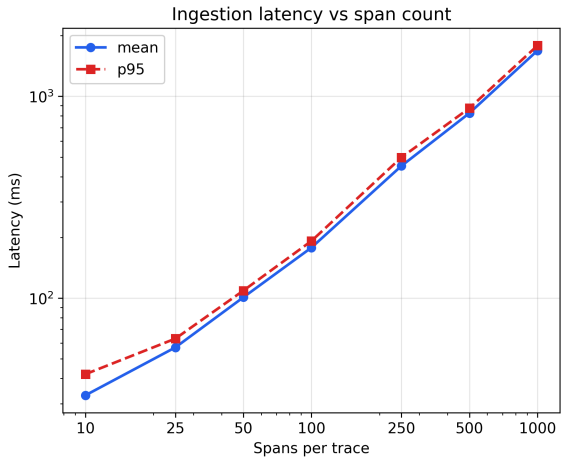
SQLite ingestion

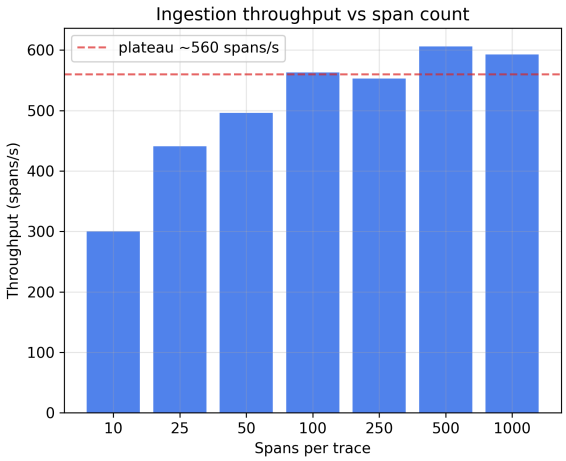
Each iteration measures one `start_trace()` plus one `log_spans()` round-trip:

```
trace_info = store.start_trace(experiment_id, timestamp, request_metadata, tags)
store.log_spans(experiment_id, trace_id, spans)
```

spans/trace	mean (ms)	p95 (ms)	traces/s	spans/s
10	33	42	30.0	300
25	57	63	17.6	441
50	101	109	9.9	496
100	178	192	5.6	563
250	452	498	2.2	553
500	826	874	1.2	606
1,000	1,686	1,784	0.6	593

- Latency is approximately linear in span count.
- Throughput plateaus around 550-600 spans/s once per-trace fixed costs are amortized.





Sustained load

Continuously ingests traces at a target QPS for 60s and measures achieved throughput, latency, and resource usage:

```
t0 = time.time()
while time.time() - t0 < 60:
    time.sleep(1 / target_qps)
    store.start_trace(...)
    store.log_spans(..., spans)
    # record latency, cpu, db_size per iteration
```

Takeaway

Under sustained write load, the system saturates sharply. Span count dominates throughput; payload size has only a modest effect.

Max throughput

spans/trace	payload	max QPS	spans/s	p50 (ms)	p95 (ms)	CPU %	DB (MB)
10	small	65.6	656	15	19	94%	46
10	100KB	57.6	576	17	21	91%	786
50	small	18.6	932	53	62	95%	1,280
50	100KB	19.3	967	51	60	94%	1,674
100	small	10.6	1,063	93	110	94%	2,258
100	100KB	10.4	1,036	96	110	95%	2,604

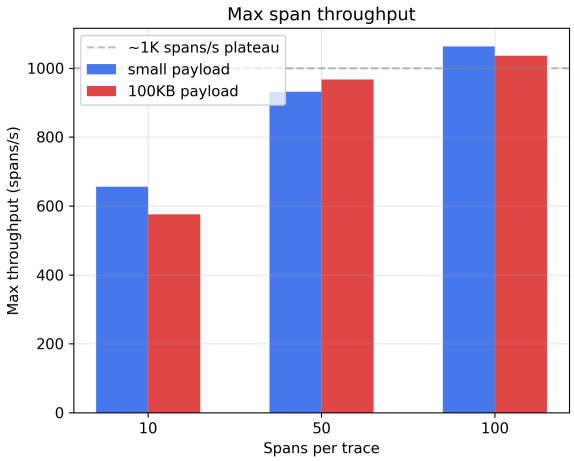
Target rates

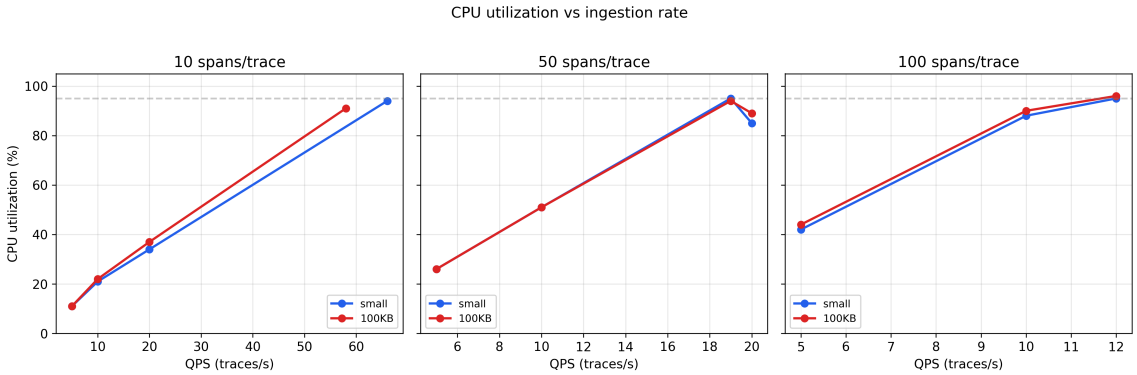
column	meaning
target	requested QPS

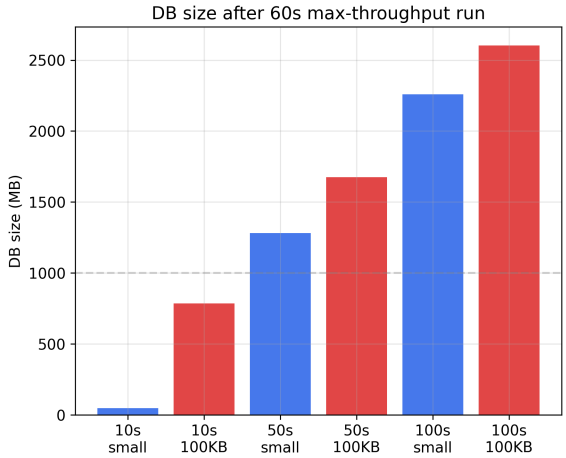
achieved QPS	actual QPS sustained over the run
headroom	`1 - (achieved / max)` — how much capacity remains. **SAT** = saturated

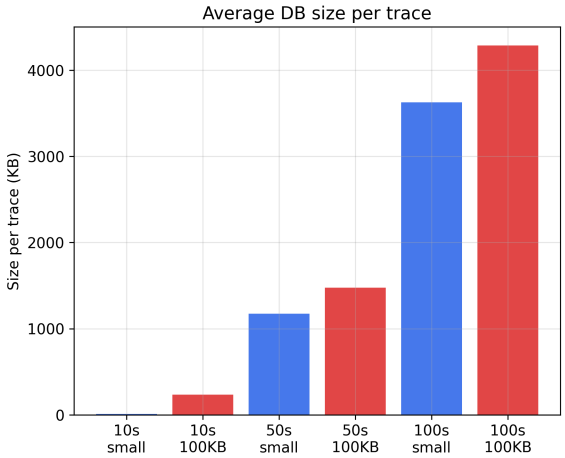
spans	payload	target	achieved QPS	p50 (ms)	p99 (ms)	CPU %	headroom
10	small	5	5.0	22	37	11%	92%
10	small	10	10.0	22	34	21%	85%
10	small	20	20.0	19	26	34%	69%
50	small	5	5.0	54	74	26%	73%
50	small	10	10.0	54	72	51%	46%
50	small	20	20.0	45	58	85%	-7%
100	small	5	5.0	89	116	42%	53%
100	small	10	10.0	92	116	88%	6%
100	small	20	11.8	84	107	95%	**SAT**
100	100KB	20	11.9	83	102	96%	**SAT**

- Span count dominates throughput.
- Payload size has modest impact on throughput but large impact on DB growth.
- SQLite saturation is CPU-bound and sharp.







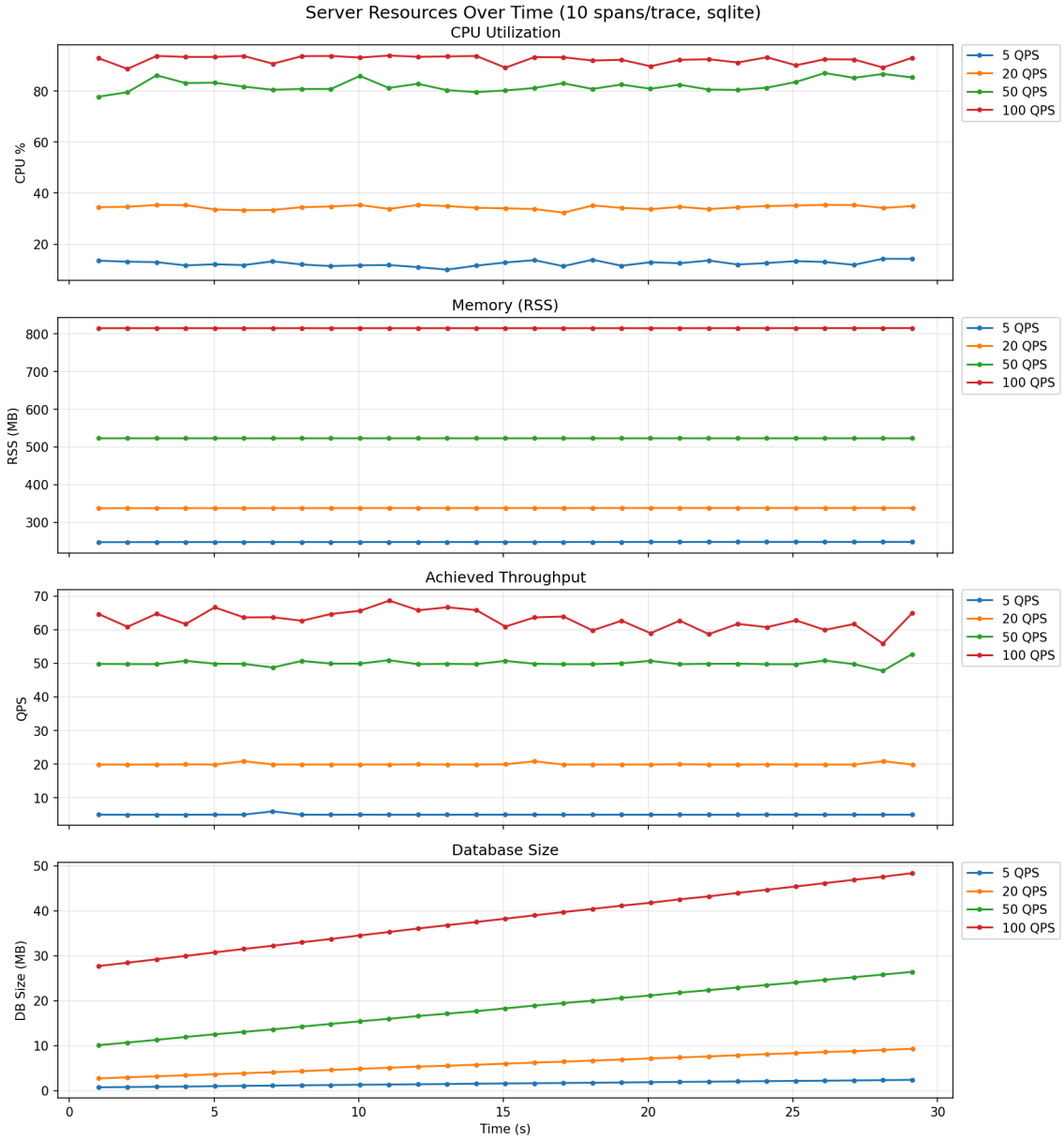


Resource usage over time

Monitoring CPU, RSS, throughput, and DB size per second at varying QPS and span counts shows:

- **CPU scales linearly with QPS** until saturation. At 10 spans: 5 QPS uses ~14%, 20 QPS uses ~35%, 50 QPS uses ~78%, 100 QPS saturates at ~95%.
- **RSS stays flat over time** within each run — no memory leaks or accumulation. The differences between QPS levels are from the pre-generated trace pool, not runtime growth.

- **Throughput is stable** below saturation. Above saturation (e.g., 100 QPS target at 10 spans, or >20 QPS at 50 spans), achieved QPS flattens regardless of target.
- **DB size grows linearly** with throughput. At 50 QPS with 10 spans, the DB grows at ~0.7 MB/s.
- **At 50 spans/trace**, 50 and 100 QPS targets both saturate at ~20 QPS. At **100 spans/trace**, everything above 10 QPS saturates at ~11 QPS.



Large span content

Each span's `content` column is padded to a target size (1MB, 5MB, 10MB), then ingested and searched:

```
span.content = json.dumps({"input": "x" * target_bytes})
store.log_spans(..., [span])  # measure ingestion
store.search_traces(experiment_id)  # measure search
```

Takeaway

Large payloads increase ingestion latency and storage footprint linearly, but do not materially affect baseline search latency.

payload	p50 ingest (ms)	p95 ingest (ms)	DB/trace (MB)
small	15.3	25.6	0.062
1MB	30.5	33.7	1.993
5MB	87.6	95.2	9.695
10MB	147.8	159.0	19.327

payload	search p50 (ms)	vs small
small	8.5	1.0x
1MB	8.5	1.0x
5MB	8.6	1.0x
10MB	8.6	1.0x

Span size (attribute count)

Varies the number of attributes per span to control serialized size:

```
span.attributes = {f"attr_{i}": f"value_{i}" for i in range(num_attrs)}
# 5 attrs → 1.2 KB/span, 200 attrs → 33.9 KB/span
```

attrs/span	avg span size	ingestion p50 (ms)	get_trace p50 (ms)	search p50 (ms)
5	1.2 KB	10.1	1.14	39.6
20	3.6 KB	11.9	1.24	39.2
50	8.6 KB	12.0	1.44	38.8
100	17.0 KB	14.9	1.59	45.5
200	33.9 KB	19.4	2.87	43.1

- Ingestion scales modestly: 1.9x slower at 200 attrs (34 KB/span) vs 5 attrs (1.2 KB/span). The ORM merge overhead still dominates over content size.
- Deserialization (`get_trace`) scales similarly: 2.5x at 200 attrs.
- Search is unaffected by span size — it doesn't load span content.

Client-Side Serialization

Profiles the client export pipeline with cProfile to identify where CPU time goes.

column	what it measures	code

json_dumps	JSON-serializing span inputs/outputs via `TraceJSONEncoder`	`dump_span_attribute_value()`
to_dict	Converting each `Span` to a Python dict	`Span.to_dict()`
to_proto	Converting each `Span` to an OTel protobuf object	`Span.to_otel_proto()`
proto_ser	Serializing the full `ExportTraceServiceRequest` to bytes	`request.SerializeToString()`
size_stats	Re-serializing spans to compute JSON byte sizes	`add_size_stats_to_trace_metadata()`

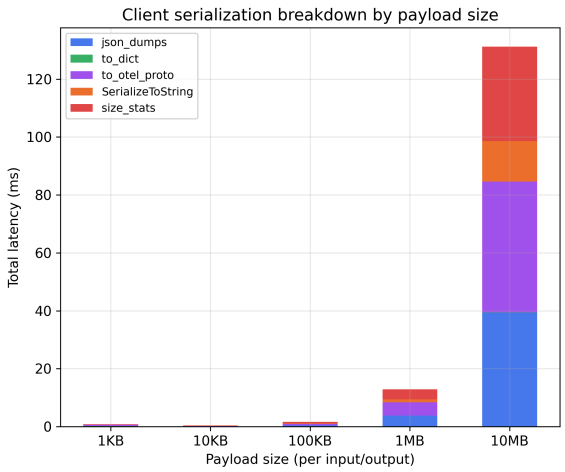
Takeaway

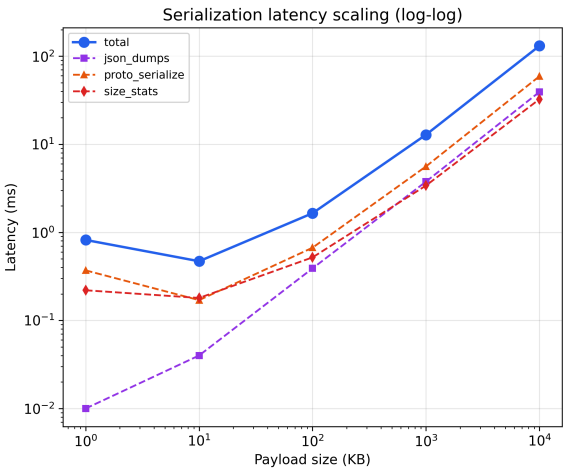
Serialization cost is linear in payload size. For large payloads, recursive OTLP conversion and redundant size-stat computation dominate client overhead.

Serialization breakdown, 10 spans/trace

payload	input KB	output KB	proto KB	json_dumps (ms)	to_dict (ms)	to_proto (ms)	proto_ser (ms)	si
1KB	1.1	1.1	4.6	0.01	0.22	0.39	0.37	
10KB	9.9	9.9	22.3	0.04	0.08	0.15	0.17	
100KB	97.9	97.9	199.5	0.39	0.07	0.52	0.67	
1MB	978.4	978.4	1971.4	3.77	0.07	4.56	5.61	
10MB	9782.9	9783.0	19689.1	39.34	0.07	45.17	59.24	

- `to_dict()` is effectively free because it reads already serialized JSON strings.
- `to_proto()` is the main CPU cost for large payloads.
- The measured `proto_serialize` number overstates true serialization cost because the benchmark redoes proto conversion inside that step.
- `add_size_stats_to_trace_metadata()` adds a second expensive pass over span data.





Search

Takeaway

Search latency is dominated by N+1 relationship loading, not the SQL query itself. With `max_results=500`, the N+1 lazy loading fires **1501 queries** (3x500+1). On top of that, full-text search and assessment filters each add their own scaling problems.

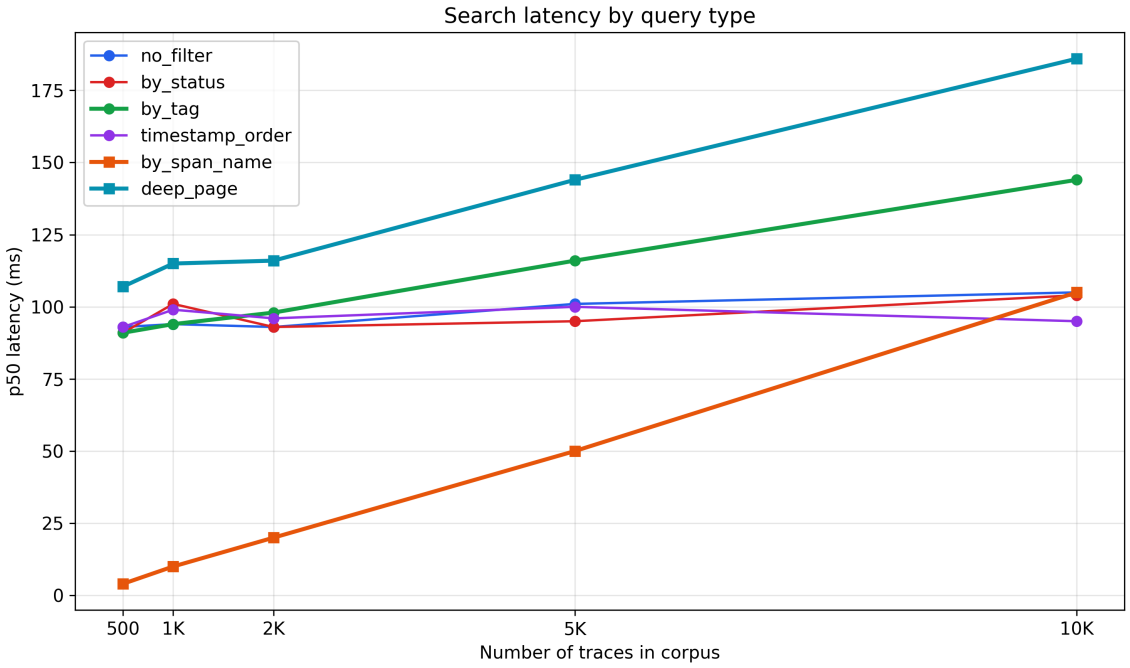
Measures `search_traces()` with common filter patterns against varying corpus sizes:

query	filter / behavior
no_filter	`""` — no filter, default sort
by_status	`"attributes.status = 'OK'"` — indexed column filter
by_tag	`"tags.env = 'prod'"` — tag table join
timestamp_order	`order_by=["timestamp DESC"]` — indexed sort
deep_page	`page_token` pointing to offset 90 — skips prior rows

Baseline search (p50 latency in ms, max_results=100)

query	500 traces	1K traces	2K traces	5K traces	10K traces
no_filter	93	94	93	101	105
by_status	91	101	93	95	104
by_tag	91	94	98	116	144
timestamp_order	93	99	96	100	95
deep_page	107	115	116	144	186

- **Flat baseline (~90-105 ms):** `no_filter`, `by_status`, and `timestamp_order` barely change with corpus size because the 301 lazy-load queries dominate. With `max_results=500` this is ~5x worse.
- **Tag filter degrades moderately:** 91 ms → 144 ms at 10K.
- **Deep pagination:** 107 ms → 186 ms, reflecting offset-based row discard.



Full-text search (trace.text ILIKE)

`trace.text ILIKE '%query%'` maps to `span.content ILIKE '%query%'` — an unindexed ILIKE over the full JSON `content` column of every span.

traces	text ILIKE p50 (ms)	status p50 (ms)	text vs status
500	53.7	44.8	1.2x
1,000	71.3	43.4	1.6x
2,000	109.4	46.0	2.4x
5,000	204.8	48.3	4.2x

- Degrades linearly, reaching 205 ms at 5K traces — 4.2x slower than indexed status filter.
- Will continue to degrade with corpus size due to scanning unindexed JSON content.

Assessment-filtered search

Assessment filters (`feedback.{name}`) are substantially slower than baseline search even at the same query count.

filter	p50 (ms)	p95 (ms)	queries	vs no_filter
no_filter	54.6	104.9	304	1.0x
feedback.correctness IS NOT NULL	339.0	393.5	304	6.2x
feedback.relevance IS NOT NULL	342.1	483.2	304	6.3x

The overhead comes from a DISTINCT subquery on the unindexed assessments table joined into the main search.

SQL query plan

`EXPLAIN QUERY PLAN` on SQLite confirms index usage for most queries:

query	scan type	index	plan detail
no_filter	SEARCH (index)	Yes	`trace_info` uses composite index
by_status	SEARCH (index)	Yes	`trace_info` uses composite index
by_tag	SEARCH (index)	Yes	`trace_tags` uses autoindex
timestamp_order	SEARCH (index)	Yes	`trace_info` uses composite index

Trace Loading (Deserialization)

Measures the cost of loading a single trace from the store:

```
trace = store.get_trace(trace_id)
# internally: SELECT content FROM spans WHERE trace_id = ?
# then for each row: json.loads(content) → Span.from_dict(parsed)
```

Takeaway

`get_trace()` deserializes every span via `json.loads(content)` + `Span.from_dict()`. This cost scales with both span count and payload size.

By span count (small payload)

spans	p50 (ms)	p95 (ms)	per-span (us)
10	1.16	1.26	116.0
50	2.23	2.39	44.6
100	3.66	4.21	36.6
250	7.16	10.23	28.7

- Per-span cost amortizes from ~116 us at 10 spans to ~29 us at 250 spans.
- A 100-span trace loads in under 4 ms — fast for single-trace views.

By payload size (10 spans)

payload	p50 (ms)	p95 (ms)	vs small
small	1.17	1.46	1.0x
100KB	1.41	1.82	1.2x
1MB	3.74	4.38	3.2x

10MB	28.82	29.94	24.5x

- Payload size dominates: 10MB traces take 29 ms to deserialize (24.5x vs small).
- The cost is `json.loads()` on the content column — linear in content size.

Batch loading

traces	p50 (ms)	p95 (ms)	per-trace (ms)
1	1.27	1.69	1.27
10	8.61	14.06	0.86
50	40.41	82.30	0.81
100	82.36	128.26	0.82

- `batch_get_traces()` scales linearly. 100 traces at 10 spans each takes ~82 ms.
- Per-trace cost is slightly lower in batch mode (~0.8 ms vs 1.3 ms) due to amortized query overhead.

Runtime Overhead

Decorator overhead

Compares the cost of calling a trivial function (`return 1`) in three scenarios:

scenario	what it measures
raw	bare function call — no decorator, baseline cost
tracing disabled	`@mlflow.trace` present but `mlflow.tracing.disable()` called
tracing enabled	`@mlflow.trace` active — includes span creation and export

Takeaway

Disabled tracing is not free, and enabled tracing is dominated by span creation and export.

scenario	p50 (us)	p95 (us)	mean (us)	vs raw
raw (no decorator)	0.08	0.13	0.10	1.0x
tracing disabled	16.38	17.38	16.64	195.0x
tracing enabled	12056	14700	12392	143534x

Streaming overhead

Measures per-chunk overhead when tracing a generator:

```
@mlflow.trace
def stream():
    for i in range(num_chunks):
        yield f"chunk_{i}"
```

```
list(stream())  # force full consumption
```

Takeaway

Tracing a generator adds about 10 us per yielded chunk, which becomes visible for token-by-token streaming.

chunks	untraced (ms)	traced (ms)	overhead (ms)	per-chunk (us)
100	0.01	1.77	1.76	17.56
1,000	0.11	10.36	10.26	10.26
10,000	1.16	95.35	94.19	9.42

Span processor contention

Measures `on_end()` throughput with multiple threads completing spans concurrently:

```
# N threads, each completing `spans_per_thread` spans
with ThreadPoolExecutor(N) as pool:
    pool.map(lambda s: processor.on_end(s), all_spans)
# on_end() acquires a lock → serialization point
```

Takeaway

The span processor introduces a meaningful serialization point under concurrency.

threads	spans	total p50 (ms)	per-span (us)	p95 (ms)	vs 1-thread
1	10	1.89	188.6	2.30	1.0x
1	50	7.61	152.2	8.27	1.0x
1	100	13.14	131.4	13.75	1.0x
4	10	7.28	728.1	14.09	4.2x
4	50	28.77	575.4	74.71	4.2x
4	100	58.98	589.8	114.03	4.5x

Async export batching

Measures the export queue's throughput at different batch sizes:

```
queue.put(span)  # enqueue one at a time
queue.flush()  # drain all pending
queue.export(batch_size=N)  # export N spans per round-trip
```

Takeaway

The queue is not the bottleneck. Batching helps, but returns diminish above roughly 50 spans per batch.

- Queue `put()` throughput: about 837K tasks/s
- `flush()` for 1000 pending tasks: 10.8 ms

batch_size	total (ms)	per-span (us)
1	10.9	10.9
10	3.7	3.7
50	2.6	2.6
128	2.6	2.6

Backend Comparison: SQLite Vs PostgreSQL

Takeaway

Both backends suffer from the same per-span ORM pattern, but they fail differently:

- SQLite becomes CPU-bound.
- PostgreSQL becomes round-trip-bound.

That makes the same architectural fix, bulk inserts instead of per-row ORM merge, even more valuable on PostgreSQL.

Ingestion

spans/trace	PG mean (ms)	SQLite mean (ms)	PG traces/s	SQLite traces/s	slowdown
10	74	33	13.4	30.0	2.2x
50	227	101	4.4	9.9	2.2x
100	383	178	2.6	5.6	2.2x

Search

query	PG 500	PG 1K	PG 5K	PG 10K	SQLite 10K
no_filter	179	245	239	295	105
by_status	183	244	256	314	104
by_tag	178	190	235	227	144
timestamp_order	187	193	261	272	95
deep_page	203	209	262	225	186

- PostgreSQL is much more sensitive to the N+1 hydration pattern because each lazy-load query incurs network round-trip overhead.

Sustained load

spans/trace	payload	target	achieved QPS	p50 (ms)	p95 (ms)	CPU %	DB (MB)
10	small	max	35.8	28	36	60%	32
10	small	10	10.0	33	41	18%	36
100	small	max	5.0	200	231	58%	66
100	small	10	4.6	201	331	58%	94

Root Causes

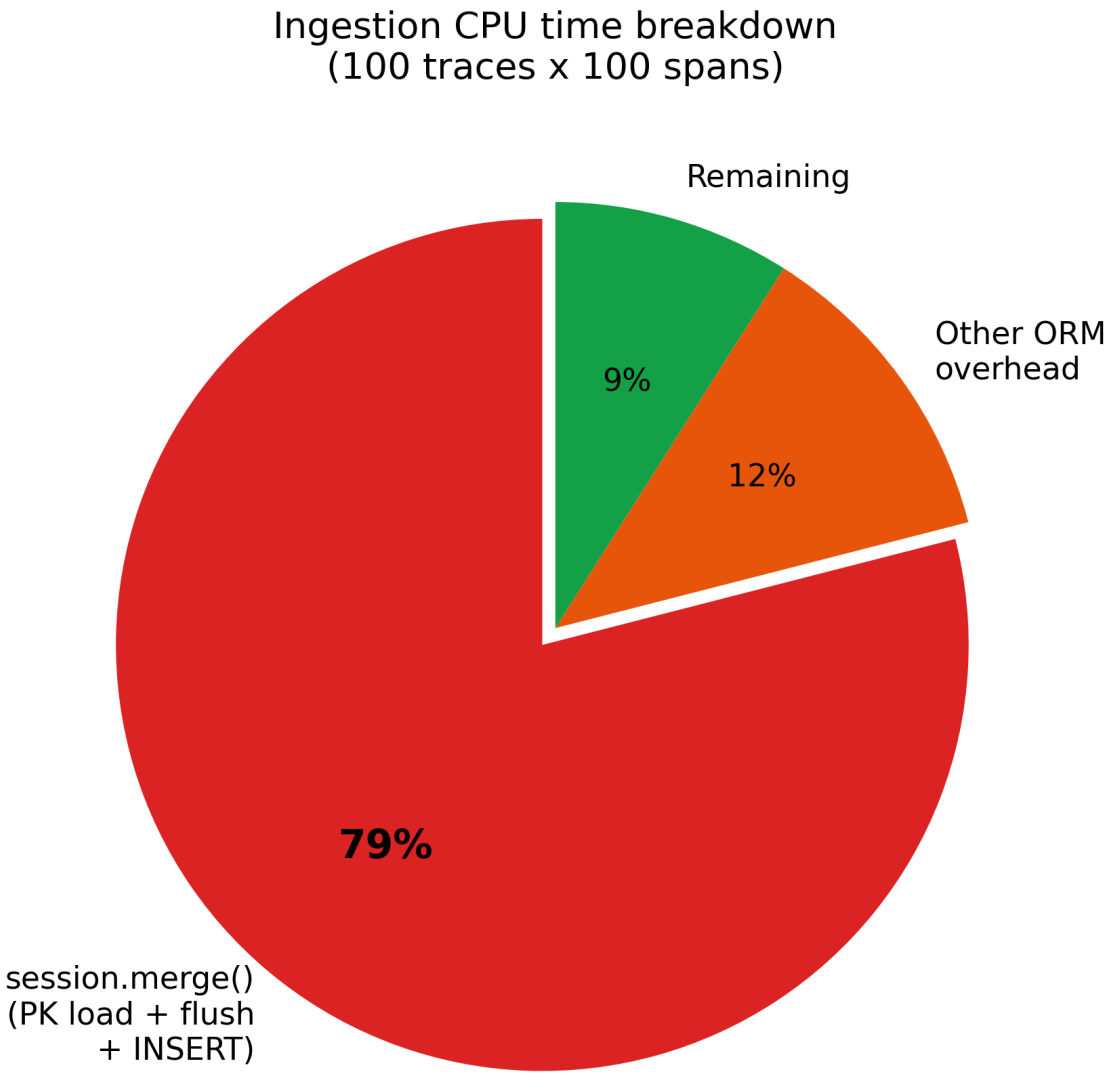
1. Ingestion write amplification in `log_spans()`

`session.merge()` is called per span and per metric. Profiling 100 traces with 100 spans shows `session.merge()` accounting for 79% of wall time.

```
ncalls  cumtime  function
19459   10.102   session.py:merge
19459    5.620   session.py:_merge
19459    4.698   session.py:_get_impl
39518    4.482   session.py:_autoflush
19659    2.429   persistence.py:save_obj
```

Potential fix direction:

```
session.bulk_save_objects(span_rows + metric_rows)
# or SQLAlchemy core INSERT ... ON CONFLICT
```



2. N+1 relationship loading in `search_traces()`

For `max_results=100`, the current pattern is effectively:

```
traces = session.query(SqlTraceInfo).limit(100).all()
for trace in traces:
    trace.tags
    trace.metadata
    trace.assessments
```

That is 301 queries for 100 traces. This explains why simple search stays near a fixed baseline and why PostgreSQL suffers disproportionately.

Potential fix direction:

```
session.query(SqlTraceInfo).options(
    joinedload(SqlTraceInfo.tags),
    joinedload(SqlTraceInfo.metadata),
    joinedload(SqlTraceInfo.assessments),
)
```

3. Full-text search scans unindexed JSON content

`trace.text ILIKE '%query%'` scans the raw JSON `content` column of every span:

```
WHERE content ILIKE '%query%'
```

This cannot use an index. `trace.text ILIKE` degrades 4.2x from 500 to 5K traces.

Potential fix direction:

- Consider a dedicated search index or pre-extracted text column

4. Offset-based pagination adds avoidable cost

Deep pagination requires the database to scan and discard prior rows:

```
ORDER BY timestamp DESC
LIMIT 10 OFFSET 90
```

Potential fix direction:

```
WHERE timestamp < :last_seen_timestamp
ORDER BY timestamp DESC
LIMIT 10
```

5. Recursive OTLP attribute conversion is expensive for large payloads

`_set_otel_proto_anyvalue()` recursively decomposes JSON structures into nested protobuf objects. At 10MB payloads this generates about 2.6M recursive calls.

Potential fix direction:

- Preserve current structure but reduce repeated conversion work
- Consider opaque string storage only if OTLP compatibility tradeoffs are acceptable

6. Size stats recompute JSON unnecessarily

`add_size_stats_to_trace_metadata()` serializes span data again purely to measure JSON byte size. This adds about 33 ms at 10MB.

Potential fix direction:

- Reuse cached JSON bytes
- Or move size computation off the hot path

7. Disabled tracing and concurrent span completion have avoidable overhead

- The tracing decorator still enters wrapper and context-manager machinery when disabled.

- The span processor uses a shared lock that serializes `on_end()` under contention.

Prioritized Recommendations

High impact

1. Bulk insert in `log_spans()`

- Expected impact: largest ingestion win, likely 5-10x in the hot path
- Why first: it addresses the dominant bottleneck on both SQLite and PostgreSQL

2. Eager-load relationships in `search_traces()`

- Expected impact: cut search query count from 301 to roughly 1-4
- Why second: it directly reduces baseline search latency and has especially high value on PostgreSQL

3. Early `is_tracing_enabled()` fast path in `@mlflow.trace`

- Expected impact: reduce disabled overhead from about 16 us toward near-zero
- Why third: low effort and immediately useful for hot paths

Medium impact

4. Indexed span content filtering

- Expected impact: remove linear scan behavior for `trace.text ILIKE` full-text search

5. Assessment-search indexing / query rewrite

- Expected impact: materially reduce the 6.2x penalty for assessment-filtered search

6. Reduce span-processor lock contention

- Expected impact: better scaling for concurrent tracing workloads

7. Batch or sample streaming events

- Expected impact: reduce generator tracing overhead for high-chunk-count streams

Lower impact

8. Collapse redundant metadata queries

- Current pattern: separate queries for token usage, cost, and session ID

9. Defer or cache size-stat computation

- Important mainly for large payloads

10. Switch deep pagination to keyset pagination

- Helpful for deeper search navigation, not the first-page experience

11. Consider opaque JSON strings in OTLP only with compatibility review

- High upside for large payload serialization
- High semantic and compatibility risk

What We Validated

Confirmed server-side issues

- Per-span and per-metric `session.merge()` in `log_spans()`
- Lazy loading of tags, metadata, and assessments in search result hydration

- Regex or text matching over JSON-backed span content
- Offset-based deep pagination
- Much slower assessment-filtered search
- Poor scaling of concurrent SQLite writes

Confirmed client-side issues

- Recursive OTLP conversion for large JSON attributes
- Redundant size-stat serialization
- Non-zero disabled tracing overhead
- Measurable streaming chunk overhead
- Span-processor lock contention under concurrency

Explicit non-bottlenecks

- HTTP transport
- Async export queue throughput
- Baseline search sensitivity to large content size

CI Guidance

Why not run benchmarks in CI today

- **Shared runners are noisy.** GitHub Actions uses shared hardware with variable load. A 10-20% swing from runner noise is indistinguishable from a real regression at the latencies we measure (~10-100 ms). This makes it hard to set meaningful pass/fail thresholds.
- **The current bottlenecks are structural.** `session.merge()` at 79% of wall time and 301 lazy-load queries per search are algorithmic problems — they won't silently regress by 5% from a code change. They either exist or they don't.

Why benchmarks alone are not enough

Even with stable hardware, wall-clock benchmarks struggle to catch **small incremental performance degradation** — the kind that accumulates over months of PRs until someone notices the system is 2x slower. Each individual PR might add 3-5% overhead that falls within measurement noise.

Better complementary approaches:

- **Query count assertions.** A test that asserts `search_traces(max_results=100)` executes exactly N SQL queries will catch any change that adds a lazy-load or an extra SELECT — regardless of hardware noise. This is the approach `bench_n_plus_one.py` uses.
- **Merge call counting.** A test that asserts `log_spans()` with 100 spans makes exactly N `session.merge()` calls will catch any regression in the ingestion path.

These are deterministic, noise-free, and can run on any CI runner.

When to revisit

- After the major bottlenecks (#1 bulk insert, #2 eager loading) are fixed, the remaining performance budget will be tighter, and marginal regressions will matter more.
- At that point, consider: (a) query-count regression tests in CI now, and (b) wall-clock benchmarks on dedicated stable hardware later.

Reproducing

```
# Full server-side benchmark
uv run python trace_perf/trace_benchmark.py

# Ingestion with cProfile
uv run python trace_perf/trace_benchmark.py --benchmarks ingest --profile

# Client-side serialization benchmark
uv run python trace_perf/bench_client_serialization.py
uv run python trace_perf/bench_client_serialization.py --profile
uv run python trace_perf/bench_client_serialization.py --spans 50

# Sustained load benchmark
uv run python trace_perf/bench_sustained_load.py
uv run python trace_perf/bench_sustained_load.py --spans 10,50 --payloads small --
qps max,10 --duration 30

# Extended store-layer benchmarks
uv run python trace_perf/bench_assessments.py
uv run python trace_perf/bench_large_content.py
uv run python trace_perf/bench_explain_queries.py
uv run python trace_perf/bench_concurrent_writers.py

# Client runtime benchmarks
uv run python trace_perf/bench_tracing_disabled.py
uv run python trace_perf/bench_streaming_overhead.py
uv run python trace_perf/bench_span_processor.py
uv run python trace_perf/bench_async_export.py
uv run python trace_perf/bench_e2e_http.py

# Generate plots
uv run python trace_perf/generate_plots.py

# Visualize profiles
uvx snakeviz trace_benchmark.prof
uvx snakeviz client_serialization.prof

# PostgreSQL
docker run -d --name mlflow-bench-pg \
  -e POSTGRES_USER=mlflow -e POSTGRES_PASSWORD=mlflow -e POSTGRES_DB=mlflow_bench \
  -p 5432:5432 postgres:16

export PG_URI="postgresql://mlflow:mlflow@localhost:5432/mlflow_bench"
uv run --with psycopg2-binary python trace_perf/trace_benchmark.py --db-uri
"$PG_URI"
uv run --with psycopg2-binary python trace_perf/bench_sustained_load.py --db-uri
"$PG_URI" --spans 10,100 --qps max,10 --duration 30
uv run --with psycopg2-binary python trace_perf/bench_concurrent_writers.py --db-uri
"$PG_URI"
uv run --with psycopg2-binary python trace_perf/bench_explain_queries.py --db-uri
"$PG_URI"
```

```
docker rm -f mlflow-bench-pg
```

Appendix: Secondary Supporting Results

Resource usage

- DB size: 217 MB for about 18.5K traces, roughly 12 KB/trace on average
- Memory: peak RSS delta about 1.7 MB, tracemalloc peak about 2 MB

Concurrent writers on SQLite

```
# N threads, each continuously ingesting traces against the same SQLite DB
with ThreadPoolExecutor(N) as pool:
    pool.map(lambda _: store.start_trace(...) + store.log_spans(...), tasks)
```

`scaling` = per-thread throughput as a percentage of single-thread throughput. 100% = perfect linear scaling.

threads	traces/s	spans/s	p50 (ms)	p95 (ms)	p99 (ms)	scaling
1	64.4	644	15.2	18.3	24.4	100%
2	67.3	673	14.4	35.6	244.3	52%
4	68.3	683	15.0	181.0	1048.6	27%
8	65.6	656	16.6	682.4	1418.5	13%

End-to-end HTTP path

```
# Direct: store.log_spans(...) bypassing HTTP
# HTTP:   POST /api/2.0/mlflow/traces with OTLP protobuf body
#         (includes client serialization + network + server deserialization)
```

spans	direct p50 (ms)	HTTP p50 (ms)	serialize (ms)	overhead
10	14.4	13.5	0.2	0.9x
50	42.2	44.0	1.0	1.0x
100	79.4	80.5	1.6	1.0x